Lecciones de la Parte I

Adaptación a C#

La sorpresa de la Parte I es lo poco que hay genuinamente nuevo cada vez. Los mismos cinco movimientos vuelven una y otra vez, aplicados en combinaciones distintas contra el mismo puñado de preguntas estructurales. La búsqueda lineal y Timsort no se parecen en nada sobre el papel; sepáralos, y ambos están leyendo la entrada un elemento a la vez y decidiendo qué hacer con lo que acaban de ver. El vocabulario del diseño de algoritmos es pequeño. Lo difícil es reconocer qué palabra aplica.

Este es el breve repaso retrospectivo. Los capítulos tienen los algoritmos; este ensayo nombra los patrones que siguieron apareciendo, las meta-lecciones que sobreviven a los detalles, y el puente hacia la Parte II, donde el foco cambia de los movimientos algorítmicos a las estructuras de datos sobre las que esos movimientos actúan.

1. Los patrones que siguieron apareciendo

El adversario

El capítulo 1 los introdujo — el jugador malicioso que puede reorganizar la entrada en el momento en que tu algoritmo no está mirando — y siguieron reapareciendo a lo largo de la parte. El adversario es el argumento de la cota inferior hecho drama, y es notablemente robusto: en el capítulo 5, el mismo adversario que forzó a la búsqueda lineal a ser $\Omega(n)$ fuerza a cualquier algoritmo de selección a ser $\Omega(n)$, porque si no lees cada elemento, el adversario pondrá el no leído a ser la respuesta. En el capítulo 6, incluso los ordenamientos en tiempo lineal respetan la cota $\Omega(n)$ del adversario. Al adversario no le importa en qué modelo estás. Solo le importa que hayas ignorado un elemento que podrías haber mirado.

Las tres preguntas

Cada capítulo de algoritmos terminó con los mismos tres subtítulos en el mismo orden: *¿Es correcto? ¿Cuán eficiente es? ¿Es óptimo?* Lo llamé el cinosura del libro en el capítulo 0, y después de pasarlo por seis capítulos de algoritmos espero que se sienta menos como una plantilla y más como una disciplina. Hacer la tercera pregunta — ¿puede algo hacerlo mejor, *en este modelo*? — es el movimiento que separa implementar un algoritmo de entenderlo.

Inversiones

Un par de elementos fuera de orden. La evidencia local de que una secuencia no está ordenada. La geometría que explica por qué el ordenamiento de burbuja y el quicksort tienen las complejidades asintóticas que tienen. El conteo de inversiones apareció como la distancia al orden en el capítulo 3, como la descomposición local-vs-global que organiza divide y vencerás en el capítulo 4, y como el efecto secundario que merge sort puede contar gratis en $O(n \log n)$. La lente es específica del ordenamiento — no se trasladará al resto del libro en la misma forma — pero la idea debajo de ella (encuentra una propiedad estructural de la entrada que el algoritmo pueda atacar incrementalmente) seguirá volviendo, con disfraces que reconocerás tras un momento de mirar.

Recursión

Todo algoritmo no trivial después del capítulo 3 fue recursivo: búsqueda binaria, merge sort, quicksort, quickselect, mediana de medianas. La recursión es cómo un programador dice "resolví el caso pequeño; el caso grande es el mismo algoritmo sobre una pieza más pequeña". Es la expresión más limpia de divide y vencerás, y el análisis de recurrencias que aprendiste a hacer — $T(n) = 2T(n/2) + O(n)$ dando $O(n \log n)$, $T(n) = T(n/2) + O(n)$ colapsando a $O(n)$ — es el álgebra del pensamiento recursivo. Las dos recurrencias difieren por un solo coeficiente. Ese coeficiente es la diferencia entre ordenamiento y selección.

Aleatorización

Quickselect fue el primer lugar donde la aleatorización se ganó su sustento, y es el ejemplo más limpio que conozco del estilo *Las Vegas*: el algoritmo siempre es correcto, pero el coste depende de una elección aleatoria que el algoritmo mismo hace. La aleatorización es una estrategia deliberada para romper patrones de entrada adversariales. Un adversario puede construir un peor caso para cualquier algoritmo determinista prediciendo sus elecciones. No puede construir un peor caso para un algoritmo aleatorizado sin predecir tu generador de números aleatorios, y no puede predecir eso.

Mirar dentro

Los primeros seis capítulos de la Parte I trataron los elementos como opacos — los algoritmos podían comparar dos de ellos pero no podían ver qué había dentro. El capítulo 6 fue la salida: el ordenamiento por conteo y el ordenamiento por base miran dentro del elemento, tratando su valor como una dirección o sus dígitos como claves separadas ordenables, y ese único cambio baja la cota inferior de $\Omega(n \log n)$ a $\Omega(n)$. El patrón recurre a lo largo del resto del libro con disfraces distintos: las tablas hash convierten claves en direcciones, los tries descomponen cadenas en caracteres, los arreglos de sufijos tratan cadenas como posiciones en un meta-arreglo.

Composición

La lección del capítulo 7, y la que va más lejos más allá de este libro. En teoría eliges el algoritmo correcto. En la práctica coses los movimientos correctos: ordenamiento por inserción para los casos pequeños, quicksort para el divide recursivo, merge sort para estabilidad, heapsort para seguridad. La habilidad del practicante no es la selección de algoritmo — es la composición de algoritmos, saber qué movimientos aplican dónde.

2. Cinco meta-lecciones

Ninguna de estas es nueva con este libro; todas son más fáciles de articular después de siete capítulos de verlas en acción.

1. La complejidad asintótica es un piso, no una receta

Saber que un algoritmo es $O(n \log n)$ no te dice si usarlo; te dice por debajo de qué no puede caer el coste, dado el modelo. El software real gana donde el modelo asintótico no ve: en efectos de caché, factores constantes, predicción de ramas, la distribución empírica de entradas reales. Timsort e introsort son algoritmos $O(n \log n)$ cuyas ganancias de ingeniería están por debajo de la resolución del orden asintótico.

2. Las cotas inferiores pertenecen a los modelos

La cota de comparaciones $\Omega(n \log n)$ del capítulo 4 no era un hecho sobre el ordenamiento. Era un hecho sobre el ordenamiento usando *solo comparaciones*. Abandona la restricción de solo comparaciones y la cota se va — el capítulo 6 fueron tres páginas demostrando exactamente eso. La lección generaliza: cada vez que veas una cota inferior, pregunta en qué modelo es una cota, y si tu problema realmente vive en ese modelo.

3. La estructura es información, y la información es lo que los algoritmos convierten en velocidad

La búsqueda lineal tiene $O(n)$ porque la entrada no está estructurada. La búsqueda binaria tiene $O(\log n)$ porque la entrada está ordenada. El ordenamiento por conteo tiene $O(n)$ porque el rango de la entrada está acotado. Cada aceleración en la Parte I vino de encontrar alguna propiedad de la entrada que el algoritmo anterior no estaba explotando. El movimiento general: ¿qué sabe la entrada sobre sí misma, que yo aún no estoy usando?

4. El algoritmo correcto rara vez es un solo algoritmo

Las rutinas de ordenamiento de producción son composiciones. El mismo patrón aparece dondequiera que se escriba código serio: las tablas hash componen encadenamiento con direccionamiento abierto dependiendo de la carga; los árboles balanceados componen rotaciones con heurísticas basadas en tamaño; los algoritmos de grafos componen BFS, DFS, y recorrido basado en prioridad dependiendo de la pregunta que se hace. El algoritmo teórico es el punto de partida. El algoritmo de producción es la composición.

5. Haz siempre la tercera pregunta

¿Es esto óptimo? ¿Puede algo hacerlo mejor, en este modelo? Hazlo un reflejo — la cosa que haces en el momento en que terminas de escribir un algoritmo. La respuesta no tiene que ser sí. El preguntar es lo que separa escribir código que funciona de entender la forma del coste del problema.

3. Lo que viene

La Parte II es un cambio de altitud. Los siete capítulos que acabas de terminar trataban de movimientos algorítmicos — divide y vencerás, explotar estructura, aleatorizar, mirar dentro, componer. La siguiente parte trata de los sustratos sobre los que esos movimientos actúan: las estructuras de datos que almacenan y recuperan datos eficientemente, para que los algoritmos de las Partes III a IX tengan sobre qué pararse.

Listas enlazadas, pilas, colas, tablas hash, montículos, árboles. Ninguna de estas es un algoritmo por sí misma; cada una es un contrato — un conjunto de operaciones con una forma de coste — sobre el que los algoritmos del resto del libro se apoyarán sin pensarlo. Las tablas hash hacen posibles los algoritmos $O(n)$ para problemas que eran $O(n \log n)$ con estructuras basadas en comparaciones. Los montículos hacen posible el algoritmo de camino más corto de Dijkstra. Los árboles hacen las consultas ordenadas logarítmicas y las operaciones balanceadas amigables con la caché.

La lente cambia, pero los movimientos no. Verás *explotar-la-estructura-intrínseca-de-la-clave* en las tablas hash. Verás *tirar la mitad con cada comparación* en los árboles balanceados. Verás *composición sobre selección* en toda estructura de datos del mundo real, donde la implementación de producción siempre es un híbrido de dos o tres ideas de libro de texto pegadas con cuidado.

Siete capítulos, cinco movimientos. Seguirán apareciendo — adelante hacia la Parte II.